

C语言与程序设计

The C and Programming

第4章 C程序的语句和流程控制

王多强

1097412466

dqwang@hust.edu.cn

华中科技大学计算机学院

本章内容

本章介绍C语言的基本语句及流程控制结构。

4.1 C语句分类

4.2 表达式语句

4.3 复合语句

4.4 条件语句

4.5 循环语句

4.6 转移语句

4.7 程序设计应用实例

4.1 C语句分类

- ◆ 由操作数和运算符组成的式子称为**表达式**（相当于“**短语**”）。
- ◆ 程序中具有完整意义的一句话称为**语句**（相当于“**句子**”）。
- ◆ 计算机程序就是由有限条语句组成**语句序列**。这些语句按顺序依次执行，完成程序规定的功能。
- ◆ C程序的语句分为两大类：
 - **声明语句**
 - **可执行语句**

声明语句：

用来声明**用户自定义数据类型**、**变量**、**函数原型**等的语句。

常见的声明语句有：

- ◆ **定义变量**的声明语句：`int x;`
- ◆ **定义外部变量**的声明语句：`extern int x;`
- ◆ **定义符号常量**的声明语句：`#define PI 3.14`
- ◆ **定义const常量**的声明语句：`const int x = 5;`
- ◆ **数据类型定义**声明语句：`enum week {SUN, MON, TUE, WED, THU, FRI, SAT};`
- ◆ **typedef类型定义**声明语句：`typedef char myChar;`
- ◆ **函数原型**声明语句：`void show(char str[ ]);`

可执行语句

用来**对数据进行处理**和**对程序流程进行控制**的语句。

可执行语句有以下六类：

- ◆ **表达式语句、复合语句**
- ◆ **选择语句** (if 语句和 switch 语句)
- ◆ **循环语句** (while 语句, do-while 语句和 for 语句)
- ◆ **转移语句** (break 语句, continue 语句, goto 语句和 return 语句)
- ◆ **标号语句** (配合 goto 语句)

4.2 表达式语句

- ◆ **表达式**：由**运算符**和**操作数**组成的式子称为**表达式**，
如： $x=y+1$
- ◆ **表达式语句**：在**表达式末尾加上分号“;”**就构成**表达式语句**。
- ◆ **表达式语句的一般形式**：**表达式**；

例：

以下称为表达式：

计算表达式 { $x = y + 1$
 $x += y$
 $i = j = k$

函数调用表达式 { `printf("hello")`
`scanf("%d", &x)`



加分号

以下称为表达式语句：

$x = y + 1;$
 $x += y;$
 $i = j = k;$ } 计算表达式语句

`printf("hello");`
`scanf("%d", &x);` } 函数调用语句

◆ **空语句**：仅由一个分号构成的语句称为**空语句**，即： **;**

- 空语句不执行任何操作，仅起到 **“行占位”** 的作用。

程序中，如果某处**在语法上需要一条语句**，而在实际功能上不需要执行任何操作时，就可以使用空语句，如空的循环体，如：`while(i--);`。

◆ **a - b;**：也是一个表达式语句

- 通常一个运算表达式要将结果赋给一个变量，由此构成**赋值语句**，如 `z=x-y;`。
- 没有赋值操作的表达式语句在语法上是合法的，只是没有改变任何变量的值，所以这样的表达式语句通常**没有实际意义**。

4.3 复合语句

- ◆ 将一组用一对花括号 “{ }” 括起来语句称为**复合语句**。
- ◆ 复合语句的基本语法：

```
{  
  
    声明部分  
  
    执行语句部分  
  
}
```

如：

```
{  
    int t;  
    t = a;  
    a = b;  
    b = t;  
}
```

← 复合语句中可以声明
属于自己的**局部变量**

复合语句形式上是“**语句块**”，在语法上**等价于一条语句**，
可以出现在任何单语句可以出现的位置

◆ **空复合语句**：不包含其他任何语句的复合语句。 **{ }**

◆ **嵌套**：一个复合语句中可以包含其它复合语句，形成**嵌套的复合语句**结构。如：

```
{  
    int a = 0, b = 1;  
    {  
        int a=1;  
        printf("a=%d\n", a);  
        printf("b=%d\n", b += 1);  
    }  
    printf("a=%d\n", a);  
    printf("b=%d\n", b);  
}
```

注：在内、外层复合语句中定义有**同名**的**两个a变量**。在不同范围内定义的变量分别独立存在，有各自不同的**作用域、可见性**。

(详见第五章)

复合语句的使用：

- ◆ 程序中任何允许出现**单语句**的地方都允许使用**复合语句**。

复合语句在语法上可以被当作一个整体看待，等同于单语句的语法存在。可以单独存在，也可以作为其它语句的子句。

- ◆ 如果计算过程用一条语句不够表示，就用包含多个语句的复合语句表示。例如 **if语句的子句**及**循环语句的循环体**等。

例4.1 输入两个整数，按从小到大的顺序输出这两个数。

分析： 定义两个变量a和b保存输入的两个整数。如果 $a > b$ ，就交换两个变量的值，否则保持不变，然后依次输出a和b。

三重嵌套

局部变量t的作用域： 临时用于变量a和b的交换，仅在定义它的那层复合语句内有效（包括里面嵌套的更深层复合语句），出了这层复合语句就不能再使用了。

```
int main(void)
{
    int a, b;
    printf("input a, b:\n");
    scanf("%d%d", &a, &b);
    {
        int t;
        if (a > b)
        {
            t = a; a = b; b = t;
        }
    }
    printf("a=%d, b=%d\n", a, b);
    return 0;
}
```

程序的基本控制结构

逻辑上可以证明，任何复杂的程序都可以由以下三种基本结构构成；

- ◆ **顺序结构**：一条语句接一条语句依次顺序执行；

- ◆ **分支结构**：根据条件，选择不同的分支执行；

具体形式：**if语句**、**switch语句**

- ◆ **循环结构**：在一定的条件下重复执行某些语句，直到条件不满足；

具体形式：**for循环语句**、**while循环语句**、**do-while循环语句**

4.4 分支结构

分支结构根据条件选择执行不同的操作。实现分支结构的语句称为**条件语句**。C语言中有**if语句**和**switch语句**两种形式。

1. if 语句

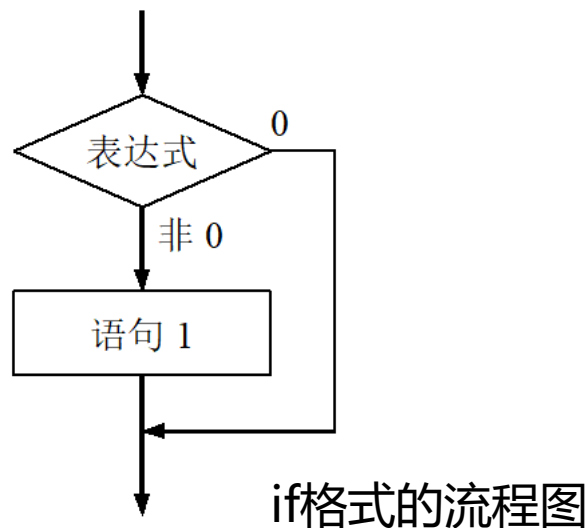
if语句有两种基本形式：

(1) 单分支结构：**if格式**

if(条件表达式)

语句1 **if子句**

含义：如果条件表达式为真，
则执行语句1，否则什
么都不做。



条件表达式：任何**值型**表达式都可以，**非0为真**，**0为假**。

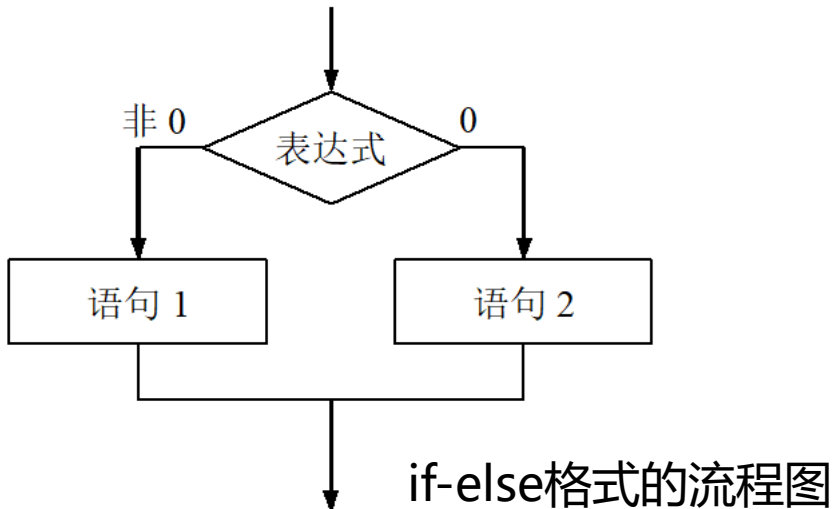
(2) 双分支结构: **if-else**格式

if(条件表达式)

语句1 if子句

else

语句2 else子句



含义：如果条件表达式为真，则执行语句1，否则执行语句2。

- 语句1称为**if子句**，语句2称为**else子句**。
- 语句1和语句2语法上要求是**单语句结构**，如果需要多条语句，则需要用 “**{ }**” 将这些语句括起来成为一条**复合语句**。

(3) if语句可以嵌套

当**if子句**或**else子句**中**又包含if 语句**时，形成**嵌套的if语句**。

例：求a, b, c 三个数中最大值（用变量**max**记录最大值）：

```
if ( a > b )
    if ( a > c ) max = a;
    else max = c;
else
    if ( b > c ) max = b;
    else max = c;
```



```
if ( a > b ) {
    if ( a > c ) max = a;
    else max = c;
}
else {
    if ( b > c ) max = b;
    else max = c;
}
```

加花括号更清晰

if-else的配对规则：

当多重嵌套if-else时，**else**与其前面距离最靠近的且还未配对的**if**配对，称为“**内层优先配对原则**”。

例4.2：学生考试成绩按以下标准分等（x为学生考试分数）：

分数范围	等级英文名
$90 \leq x \leq 100$	excellent (优)
$80 \leq x < 90$	good (良)
$60 \leq x < 80$	middle (中)
$x < 60$	bad (差)

输入某学生的考试分数，输出该学生成绩的英文等级。

分析：

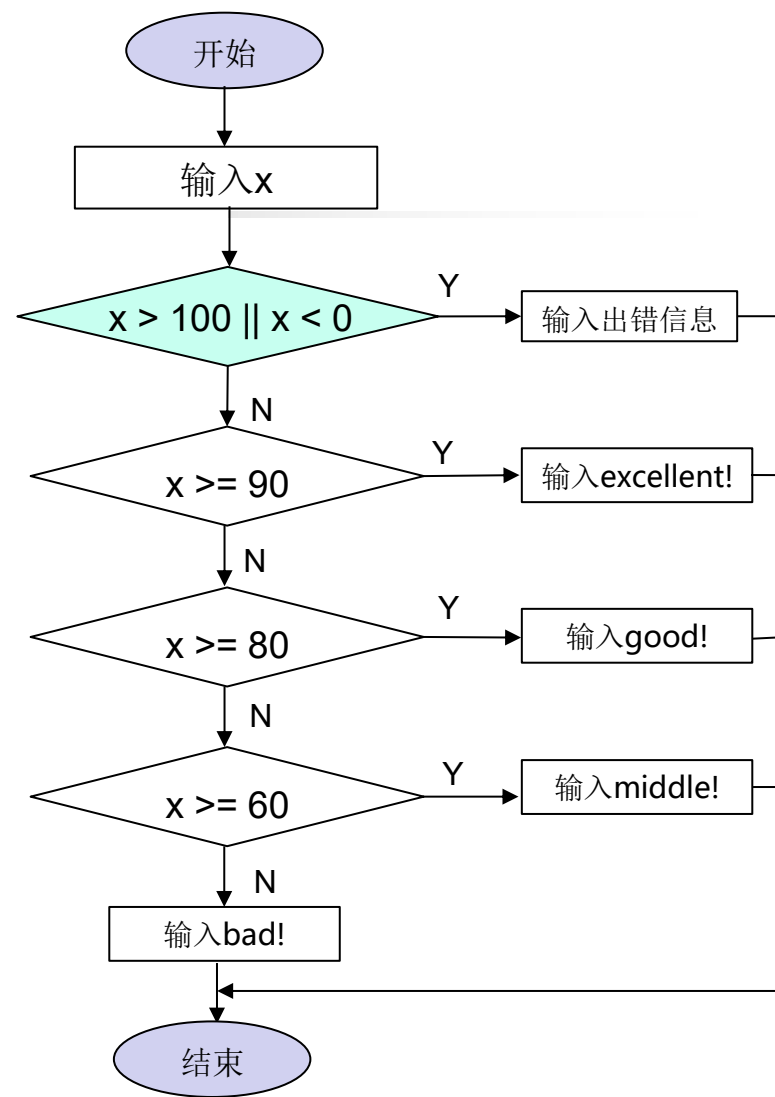
由于要**根据学生成绩的不同区间决定输出不同的内容**，所以要用分支语句。而且有多个区间，所以要用**嵌套的if-else语句**来处理不同的情况。

```

#include <stdio.h>

int main(void)
{
    float x;
    printf("input the score x( 0 <= x <= 100):\n");
    scanf("%f", &x);
    if ( x > 100 || x < 0 )
        printf("input error!\n");
    else if ( x >= 90)
        printf("excellent! \n");
    else if ( x >= 80)
        printf("good! \n");
    else if ( x >= 60)
        printf("middle! \n");
    else printf("bad! \n");
    return 0;
}

```



算法流程图

2. switch语句

switch语句是一种**多分支语句**。语句的一般形式:

switch(表达式) { ← 这个表达式必须为**整型表达式**

case 常量表达式1: 语句序列1;

case 常量表达式2: 语句序列2;

...

case 常量表达式n: 语句序列n;

default: 语句序列n+1;

}

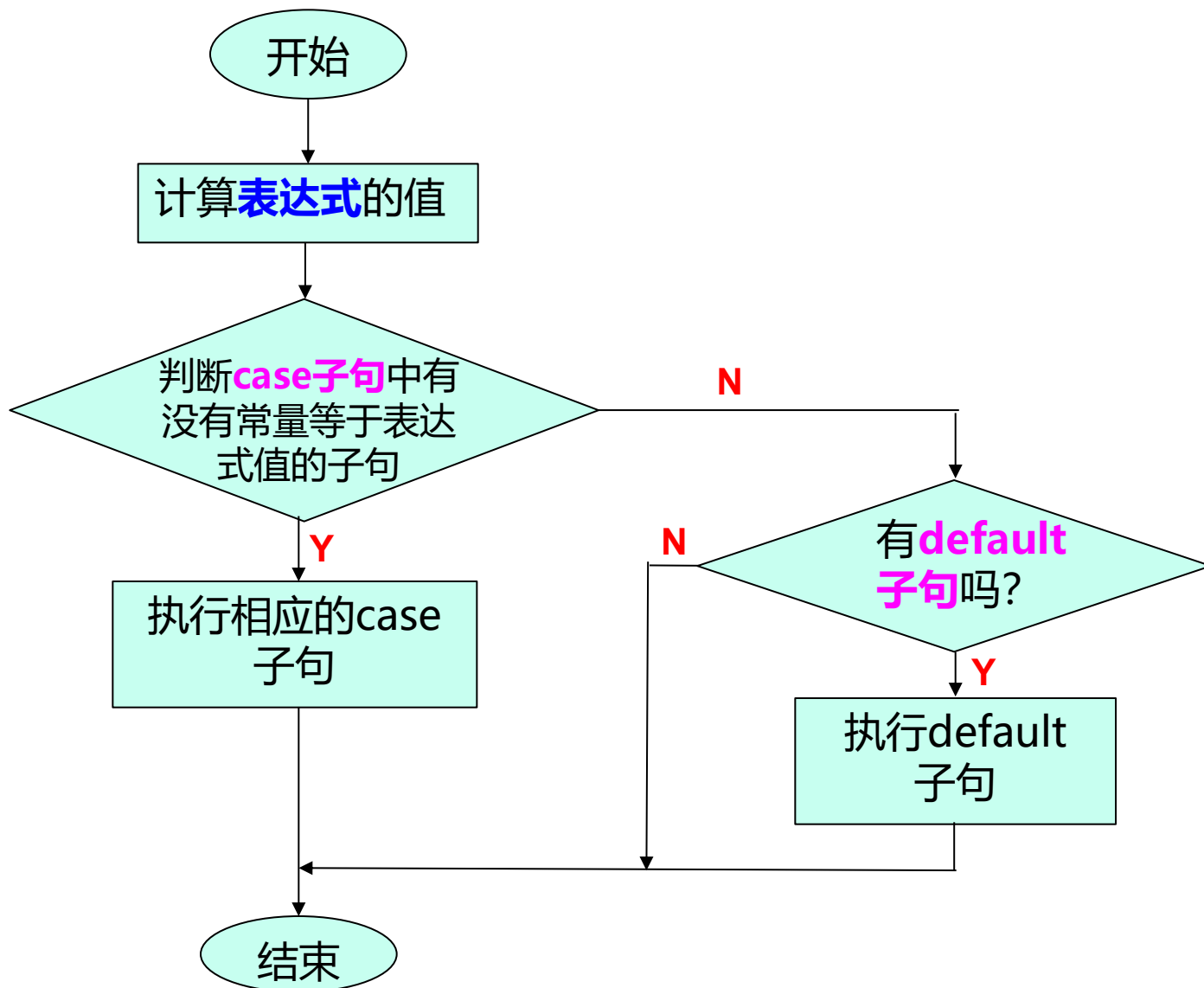
case子句

default子句

default子句
是**可选的**

这里所有**常量表达式**的值必须为**整型常量**

switch语句的一般执行流程：



转移语句：break语句

◆ 情况分析：

考虑以下switch语句程序段：

```
int i=1;
```

```
switch ( i ){
```

```
    case 0: printf("%d\t", i);
```

```
    case 1: printf("%d\t", i++);
```

```
    case 2: printf("%d\t", i++);
```

```
    case 3: printf("%d", i++);
```

```
    default: printf("\n");
```

```
}
```

```
printf("%d\n", i);
```

观察输出结果：

1 2 3 ✓
4

现象： i 不止被加和输出了1次，从 case 1开始往下的所有语句都被执行了。什么原因？

case子句的执行不会自行终止，必须用break语句强行终止。

break语句：一种**转移语句**，可以用在switch语句中的case子句中，当执行到break语句时**跳出整个switch语句**，转而执行switch语句后面的其它语句。

上述程序的改造：

```
int i=1;
switch ( i ){
    case 0: printf("%d\t", i);
    case 1: printf("%d\t", i++);
    case 2: printf("%d\t", i++);
    case 3: printf("%d", i++);
    default: printf("\n");
}
printf("%d\n", i);
```



```
switch ( i ){
    case 0: printf("%d\t", i);      break;
    case 1: printf("%d\t", i++);  break;
    case 2: printf("%d\t", i++);  break;
    case 3: printf("%d", i++);    break;
    default: printf("\n");
}
printf("%d\n", i);
```

输出结果：

1 2

- ◆ 如果希望执行完某个case子句之后便跳出switch语句，则必须使用**break**或**return**转移语句。
 - **break语句**：**跳出switch语句**，然后继续执行switch语句后面的语句（如果有）；
 - **return语句**：**立即结束函数**（如果是主函数，则结束程序）。
- ◆ case子句针对表达式不同的结果进行处理，所以**应列出需要处理的所有可能的情况**。除列出的值外，如果还有其它情况考虑，可使用**default子句**进行处理。如果没有default，则除case子句列出的情况外，其它都不予处理。

switch与if语句的不同：除语句形式不一样外，它们最大的不同在于**switch语句不能直接处理区间**。

例4.3：学生考试成绩按以下标准分等级（x为学生考试分数）：

分数范围

$90 \leq x \leq 100$

$80 \leq x < 90$

$60 \leq x < 80$

$x < 60$

等级英文名

excellent (优)

good (良)

middle (中)

bad (差)

输入某学生的考试分数，输出该学生的考试成绩的英文等级。

要求用switch语句来实现。 怎么实现？

switch语句中的表达式只能是“**整数值**”，不能表示区间范围，原if语句中的区间描述无法用switch语句直接实现。

```
if ( x > 100 || x < 0 )  
    printf("input error!\n");  
else if ( x >= 90)  
    printf("excellent! \n");
```

Vs.

```
switch(表达式) {  
    case 常量表达式1: 语句序列1;  
    case 常量表达式2: 语句序列2;  
}
```

← 这个表达式必须为整型表达式

case子句

怎么将分数转换成等级？ 离散化

等级 = 分数/10

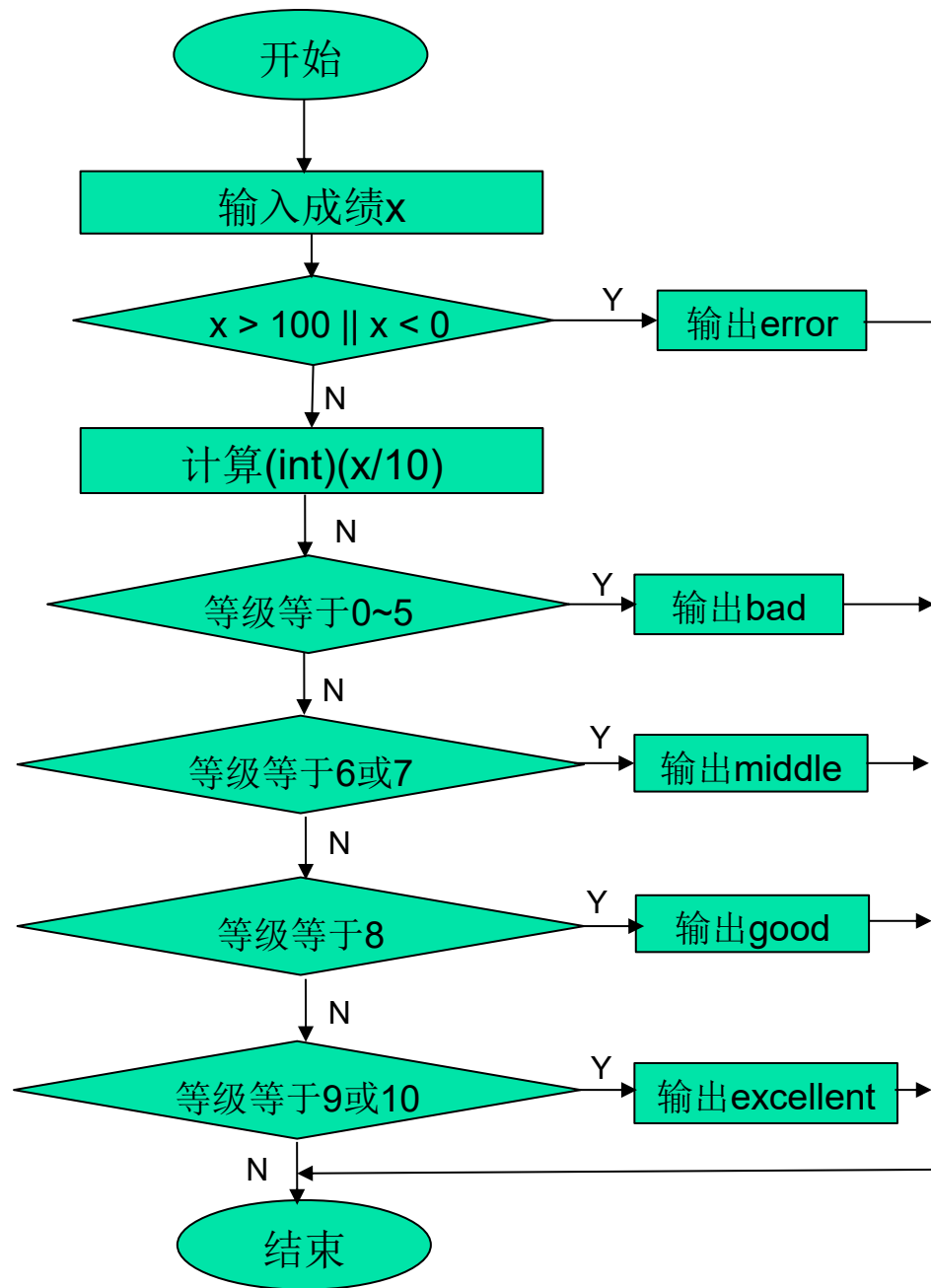
结果为整数0~10，
代表相应的分数等级

等级0~5: bad 等级6~7: middle 等级8: good
等级9~10: excellent 其它: 出错

```

#include <stdio.h>
int main(void)
{
    float x;
    printf("input the score x( 0 <= x <= 100):\n");
    scanf("%f", &x);
    if ( x > 100 || x < 0)
        printf("input error!\n");
    else
        switch ( (int)(x / 10) )
        {
            case 10: printf(" excellent! \n"); break;
            case 9:  printf(" excellent! \n"); break;
            case 8:  printf(" good! \n"); break;
            case 7:  printf(" middle! \n"); break;
            case 6:  printf(" middle! \n"); break;
            case 5:  printf(" bad! \n"); break;
            case 4:  printf(" bad! \n"); break;
            case 3:  printf(" bad! \n"); break;
            case 2:  printf(" bad! \n"); break;
            case 1:  printf(" bad! \n"); break;
            case 0:  printf(" bad! \n"); break;
        }
    return 0;
}

```



```

#include <stdio.h>
int main(void)
{
    float x;
    printf("input the score x( 0 <= x <= 100):\n");
    scanf("%f", &x);
    if ( x > 100 || x < 0) printf("input error!\n");
    else
        switch ( (int)(x / 10) )
        {
            case 10:
            case 9: printf(" excellent! \n"); break;
            case 8: printf(" good! \n"); break;
            case 7:
            case 6: printf(" middle! \n"); break;
            case 5: case 4: case 3: case 2:
            case 1: case 0: printf(" bad! \n"); break;
        }
    return 0;
}

```

对于**处理方式相同**的**case子句**，可以**共享处理语句**，写成简洁的合并形式，**共享的处理语句**写在每部分的最后。但**每个case要单独写，且冒号不能少**。

注：不能写成 case0~5: 的形式

4.5 循环语句

程序中，有些语句需要在**一定的条件下**重复执行多次，称为**循环**。循环结构用**循环语句**实现。C语言有三种循环语句形式：

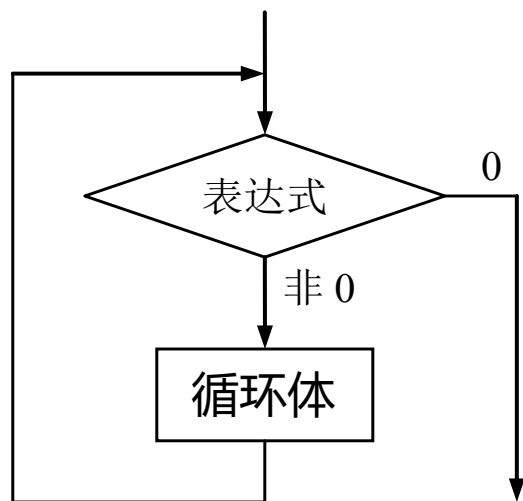
while语句、do-while语句、for语句

1. while语句

while语句的一般形式：**while (表达式)**

循环体

循环体是一条可执行语句或复合语句



while语句的流程图

首先判断**表达式**的真假。若为假，则转向while语句之后执行其它语句。若为真，则执行一次循环体，然后再次判断表达式的真假，还为真则再执行一次循环体，直到表达式为假时退出循环（即不再执行循环体，转而执行while语句之后的其它语句）。

例4.4 while循环语句示例

```
int main( )
{
    int i;
    i = 0; //初始化i
    while (i < 5) {
        printf("i=%d\t", i);
        i++; //改变i的值
    }
    printf("i=%d\t", i);
    return 0;
}
```

输出结果:

i=0 i=1 i=2 i=3 i=4 **i=5**

程序中用来**控制循环的变量**通常称为**循环控制变量**，简称"**循环变量**"。如这里的i。

while循环的要点:

while循环开始之前要对循环变量初始化，然后进入while语句。每次判断表达式是否为真，为真则循环体就被执行一次，为假则退出循环。在每次执行循环体的过程中，**要有适当的操作来改变循环变量，使表达式的值有所变换**，以此来控制循环是继续执行还是结束。

死循环：无法停止的循环（因为某种原因表达式永远为真）

如：i = 0;

```
while (i < 5) {
```

```
    printf("i=%d\t", i);
```



没有对i修改

```
}
```

```
printf("i=%d\n", i);
```

如：i = 0;

```
while (i < 5) {
```

```
    printf("i=%d\t", i);
```

```
    i++;
```

```
    i = i % 5;
```

```
}
```

```
printf("i=%d\n", i);
```

这个改变无法使i >= 5成立

注：死循环使循环无法停止，也就不能使程序最终结束。所以一般要尽量避免死循环的出现。

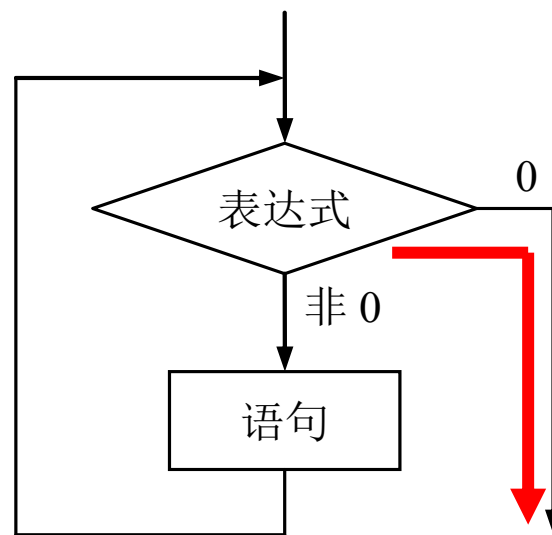
如果循环控制表达式的值一开始就为假，则while循环将被跳过，循环体一次都不会执行。

如： **i = 6;**

```
while (i < 5) {  
    printf("i=%d\t", i);  
    i++;  
}  
printf("i=%d\n", i);
```

输出结果：

i=6



2. 程序设计举例

例4.8 将来自**标准输入文件 (stdin)** 的正文复制到**标准输出文件 (stdout)**，每次输入和复制一个字符。

解析：

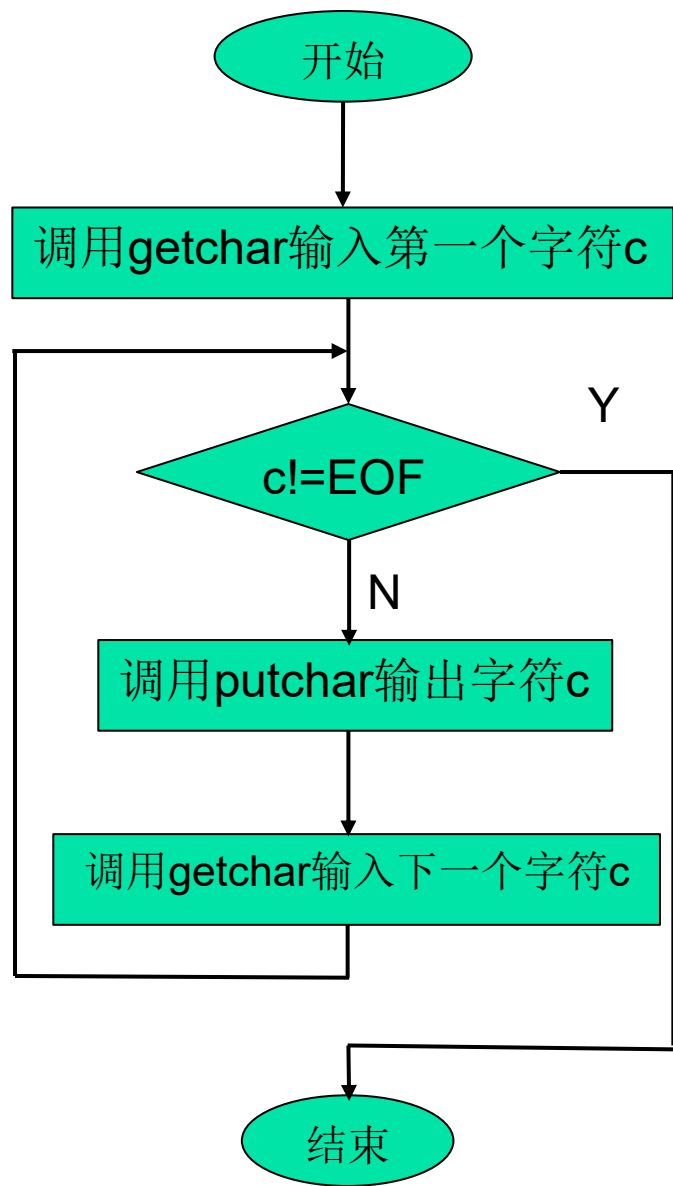
- ◆ 标准输入文件通常指键盘，标准输出文件通常指屏幕；
- ◆ 用**getchar**输入字符，用**putchar**输出字符；
- ◆ **使用循环语句**，反复调用getchar函数和putchar函数；
- ◆ **结束条件**：按下**Ctrl+Z键**，此时getchar得到**EOF**。然后结束循环。

```

#include <stdio.h>

int main(void)
{
    char c;
    printf("input a text end of ctrl + z:\n");
    c = getchar();  这里要先读入第一个字符
    while (c != EOF) {
        putchar(c);
        c = getchar();  读入下一个字符
    }
    return 0;
}

```



本程序的变形请参见教材94页，理解不同的书写形式。

例4.9 对于任意大于1的自然数 n ($n \leq 109$), 若 n 为奇数, 则将 n 变为 $3n+1$, 否则变为 n 的一半, 经过若干次这样的变换后, 一定会使 n 变为1。如:

$3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

设计一个程序, 对输入的 n , 输出上述变换的次数, 如, $n=3$, 变换7次, 输出7。

```
#include <stdio.h>
int main(void)
{
    int n, count=0;
    scanf("%d", &n)
    while (n > 1) {
        if(n%2==1) n=3*n+1;
        else n/=2;
        count++;
    }
    printf("%d\n", count);
    return 0;
}
```

count是计数器

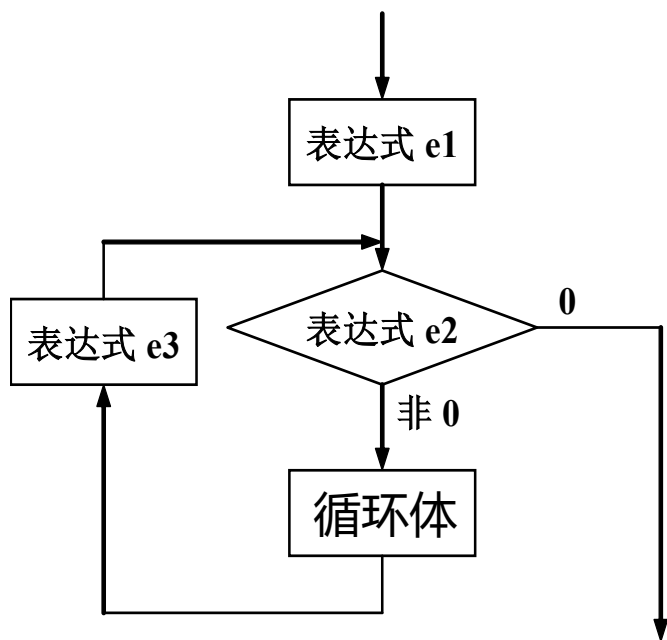
2. for语句

for语句的一般形式:

for(e1; e2; e3) 循环体s

如: **for**(i=1; i<4; i++)
 s = i*2;

循环体是一条可执行语句或复合语句



for语句流程图

for语句的控制部分被分号分成三子部分(子表达式) :

e1: 开始执行for语句时, 先执行一次e1, 所以**e1**一般用于循环变量的初始化。

e2: 循环控制条件的判断, 为真则执行一次循环体s, 为假则立即结束循环。

e3: 每执行一次循环体后, e3会被自动执行一次, 所以一般在**e3**修改循环变量的值。

(1) 三个子表达式可以全部或部分省略，但无论缺省e1，e2或e3，它们之间的**分号**不能省。

如：缺省e1和e3时的for语句形如：**for(; e2 ;) s**

缺省e2时的for语句形如：**for(e1; ; e3) s**

注：缺省e2时，默认循环条件为“永真”

(2) **e2**可以是任何形式的表达式，表达式的值非0表示真，0表示假。真时执行一遍循环体，假时结束循环。

(3) **循环体可以为空**，此时写一个分号，表示**空语句**。如：

for(i=0;i<10;i++) ;

例，设有for循环语句

```
int i;  
for(i=1; i<4; i++)  
    printf("i=%d s=%d\n", i, 2*i);
```

以下几种形式等价：

```
i=1;  
for( ; i<4; i++)  
    printf("i=%d s=%d\n", i, 2*i);
```

没有在e1部分对i初始化，就要
在for语句之前对i初始化。

```
for( i=1; i<4; ){  
    printf("i=%d s=%d\n", i, 2*i);  
    i++;  
}
```

没有在e3部分修改i，就要**在循环体中修改i。**

```
i=1;  
for( ; i<4; ) {  
    printf("i=%d s=%d\n", i, 2*i);  
    i++;  
}
```

既没有在e1部分对i初始化，也没有在e3部分修改i，那么就要在for语句之前初始化i，并循环体中修改i。

```
i=1;  
for( ; ; ) {  
    printf("i=%d s=%d\n", i, 2*i);  
    i++;  
    if (!(i<4)) break;  
}
```

① 既没有在e1部分对i初始化，也没有在e3部分修改i，那么就要在for语句之前初始化i，并循环体中修改i。

② 没有在e2部分判断循环条件（默认为永真），就要在循环体中单独判断，否则会陷入死循环。

例：输入一批整数，**以0为结束**。输出其中最大的一个值。

分析：

- 因为要输入一批整数，所以要用**循环方式**获取。
- **设置两个变量x和max**：x用于保存当前输入的数，max用于保存当前最大的那个数。

算法步骤：

- (1) 输入第一个数x。
- (2) 置max的初始值为x。
- (3) 如果x不等于0，则执行(4)~(6)；否则，转(7)。
- (4) 输入下一个数x。
- (5) **如果 $\text{max} < x$ ，则 $\text{max} = x$ 。**
- (6) 转步骤(3)。
- (7) 输出max，结束。

```
#include <stdio.h>

int main(void)
{
    int x, max;
    printf("input a group of integers end of 0: \n");
    scanf("%d", &x); /*输入第一个整数*/
    max = x;
    for( ; x!= 0 ; ) { //在输入的x不等于0的条件下循环
        scanf("%d", &x); /*输入下一个整数*/
        if (max<x)
            max = x; /*max记录最大值*/
    }
    printf("max=%d\n", max);
    return 0;
}
```

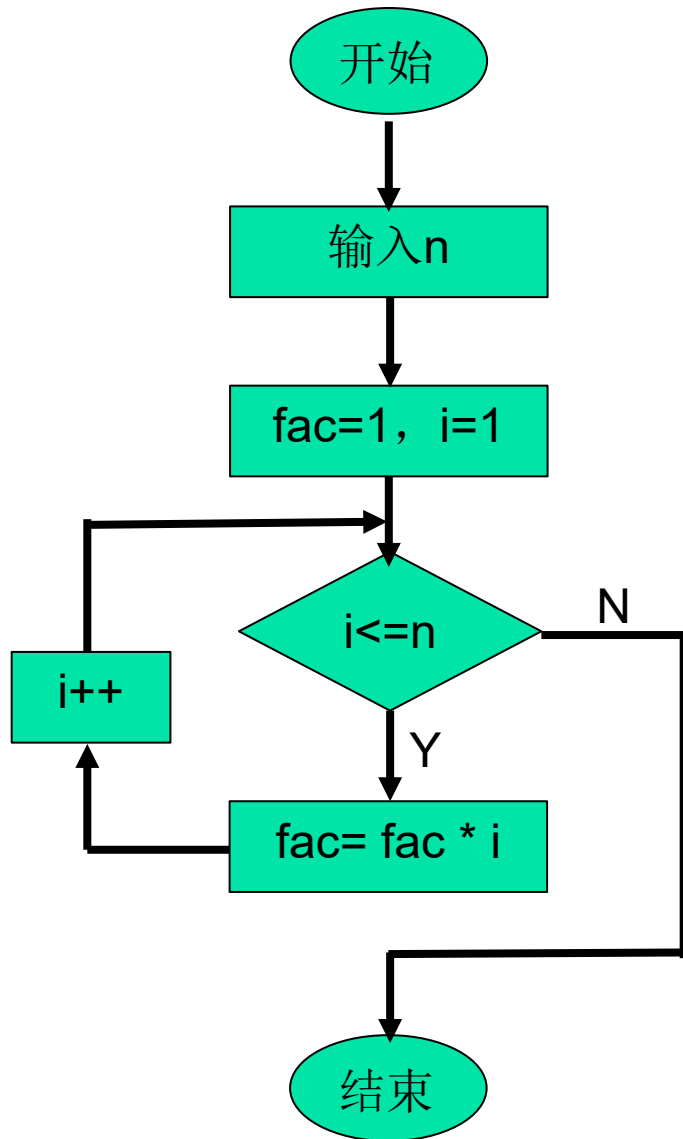
例：求 $n!$ ， n 从终端输入。

分析：

- ◆ 数学表示： $n! = 1 * 2 * 3 * \dots * n$ 。
- ◆ 程序实现：通过**循环语句**来实现**累乘**
 - 设置**循环变量** i ，并置 i 的初值为1；
 - 设置**乘积变量** fac ，初值为1；
 - **当 $i \leq n$ 时，循环：每次将 i 乘到 fac 上；**
 - **然后 $i++$ ；**
 - 循环结束条件： $i > n$ 。

注：考虑到 n 较大时， $n!$ 是一个相当大的数，为避免溢出，将 fac 变量说明为`long`，`unsigned long`，或`double`。

算法流程图：



```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int n, i;
```

```
    unsigned long fac;
```

```
    printf("input n:\n");
```

```
    scanf("%d", &n);
```

```
    for (fac=1, i=1; i<=n; i++)
```

```
        fac *= i;
```

```
    printf("%d!=%lu\n", n, fac);
```

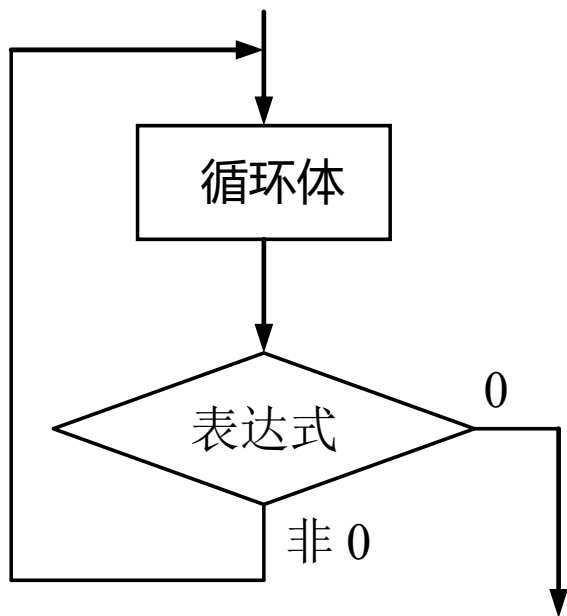
```
    return 0;
```

```
}
```

书上的例4.17、4.18课下自学

3、do-while语句

do-while语句的一般形式：**do** 循环体 **while** (表达式);



do-while语句流程图

do-while语句的执行流程：

先执行一遍循环体，然后判定表达式的值：若表达式为真，则再执行一次循环体，否则结束循环，转而执行while后面的其它语句。

特点：在判定条件之前循环体先执行一遍

例4.11：把输入的整数按反方向输出。例如，输入的数是**12345**，则输出结果是**54321**。

分析：

- 对输入的数，按**个、十、百、千、万**...位的顺序拆解各位的数码依次输出即可。

- **求取各位数码的方法：**（设整数变量为x）

第一步： **$x \% 10$** 得个位数数码。

第二步： **$x = x / 10$** 去掉现有的个位数，并形成新的个位数。

然后重复上述步骤，即可以从低位到高位得到原数各位的数码，直到x等于0，算法结束。

```
#include <stdio.h>
```

```
int main(void)
```

```
{ int x;
```

```
printf("input an integer:\n");
```

```
scanf("%d", &x);
```

```
do
```

```
    putchar(x%10 + '0');
```

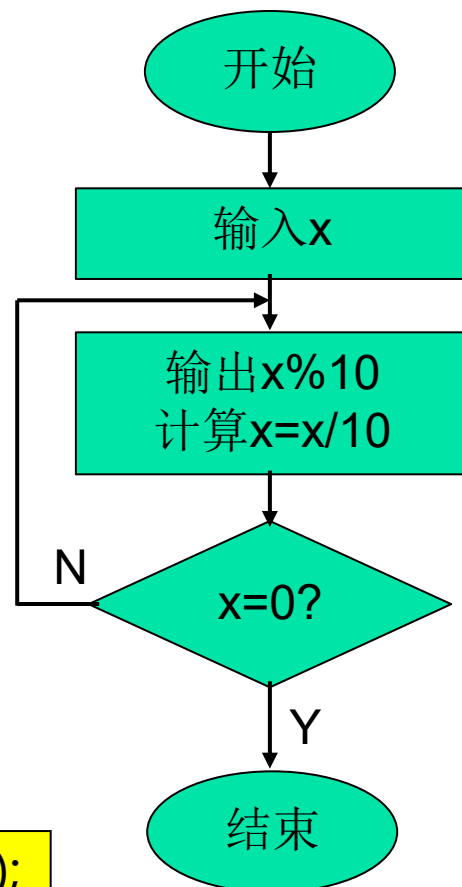
```
while ((x/=10) != 0);
```

```
printf("\n");
```

```
return 0;
```

```
}
```

也可以用 `printf("%d", x%10);`



*例4.12 用牛顿法解方程（**自学**）

例4.13 输入一个大于或等于2的整数 n ，判断该数是否为素数。

素数的判定规则：

- (1) 2是最小素数；
- (2) 一个大于2的整数 n ，若除1和 n 外，不能被其它任何数整除，则 n 是素数。

算法：用2至 $\sqrt{n}$ 之间的整数除 n 。如果均不能整除 n ，则 n 是素数，否则 n 不是素数。

```
#include <stdio.h>
```

```
#include <math.h>    /*下面用的sqrt函数在math.h头文件中*/
```

```
int main(void)
```

```
{
```

```
    int n, i, k, r;
```

```
    do { printf("input n:\n"); scanf("%d", &n); } while (n < 2);    /*输入一个不小于2的整数*/
```

```
    if (n == 2) printf("2 is a prime\n");
```

```
    else if (n > 2) {
```

```
        i = 1;
```

```
        k = sqrt(n);        /*sqrt: 求平方根的函数, 计算n的平方根*/
```

```
        do {
```

```
            ++i;
```

```
            r = n % i;    /*计算n除以i的余数*/
```

```
        } while (r && i <= k);    /*r非0表示n没有被k整除*/
```

```
        if(r) printf("%d is a prime\n", n);    /*如果到最后都没有被整除, 则n是素数*/
```

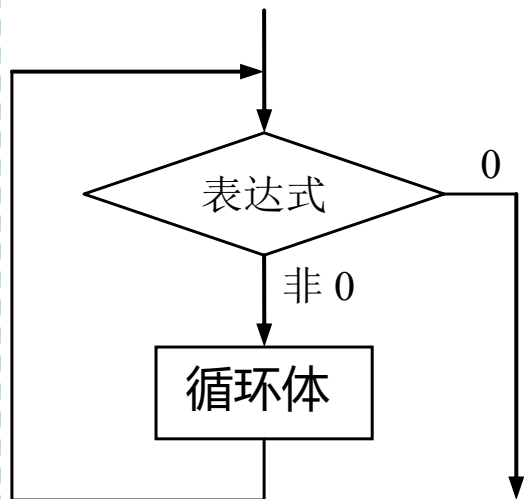
```
        else printf("%d isn't a prime\n", n);
```

```
    }
```

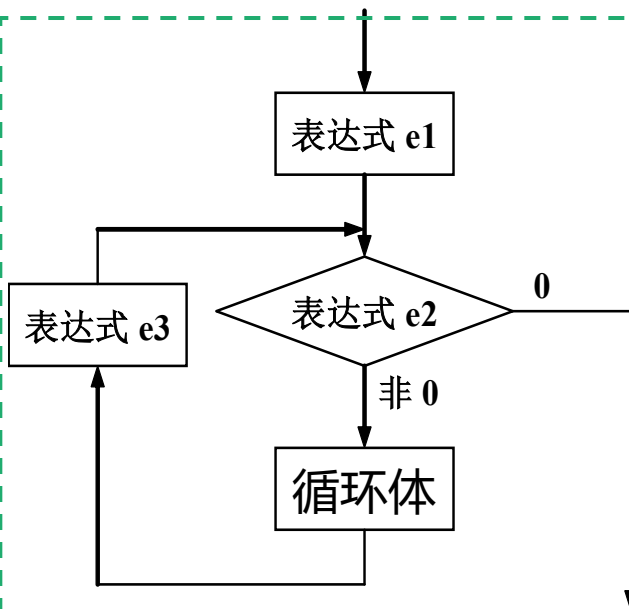
```
    return 0;
```

```
}
```

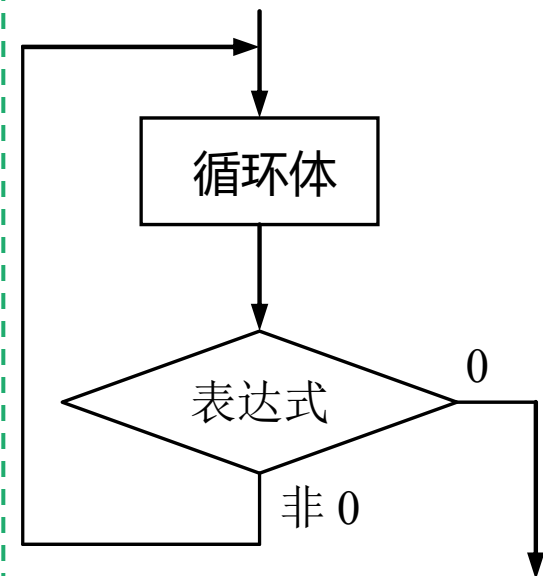
三种循环形式都可以相互等价转换



while语句流程图



for语句流程图



do-while语句流程图

```
while (表达式)
{
    循环体
}
```

```
e1;
while(e2) {
    循环体;
    e3;
}
```

```
循环体
while (表达式)
{
    循环体
}
```

4.6 转移语句

转移语句：用于改变程序的执行流程。

主要有：**break**语句、**continue**语句和**return**语句。

针对的问题：

- ◆ **情况1**：本次循环不做了，立即开始下一次循环；
- ◆ **情况2**：整个循环都不做了，立即结束并退出循环。

处理方法：

- ◆ **情况1**：用 **continue** 语句，结束本次循环，“继续”下一次循环；
- ◆ **情况2**：用**break**语句，立刻结束并退出循环。

1、break语句和continue语句

语句形式：**break;** 和 **continue;**

例 4.19 打印2~100之间的所有素数，每行输出10个数。

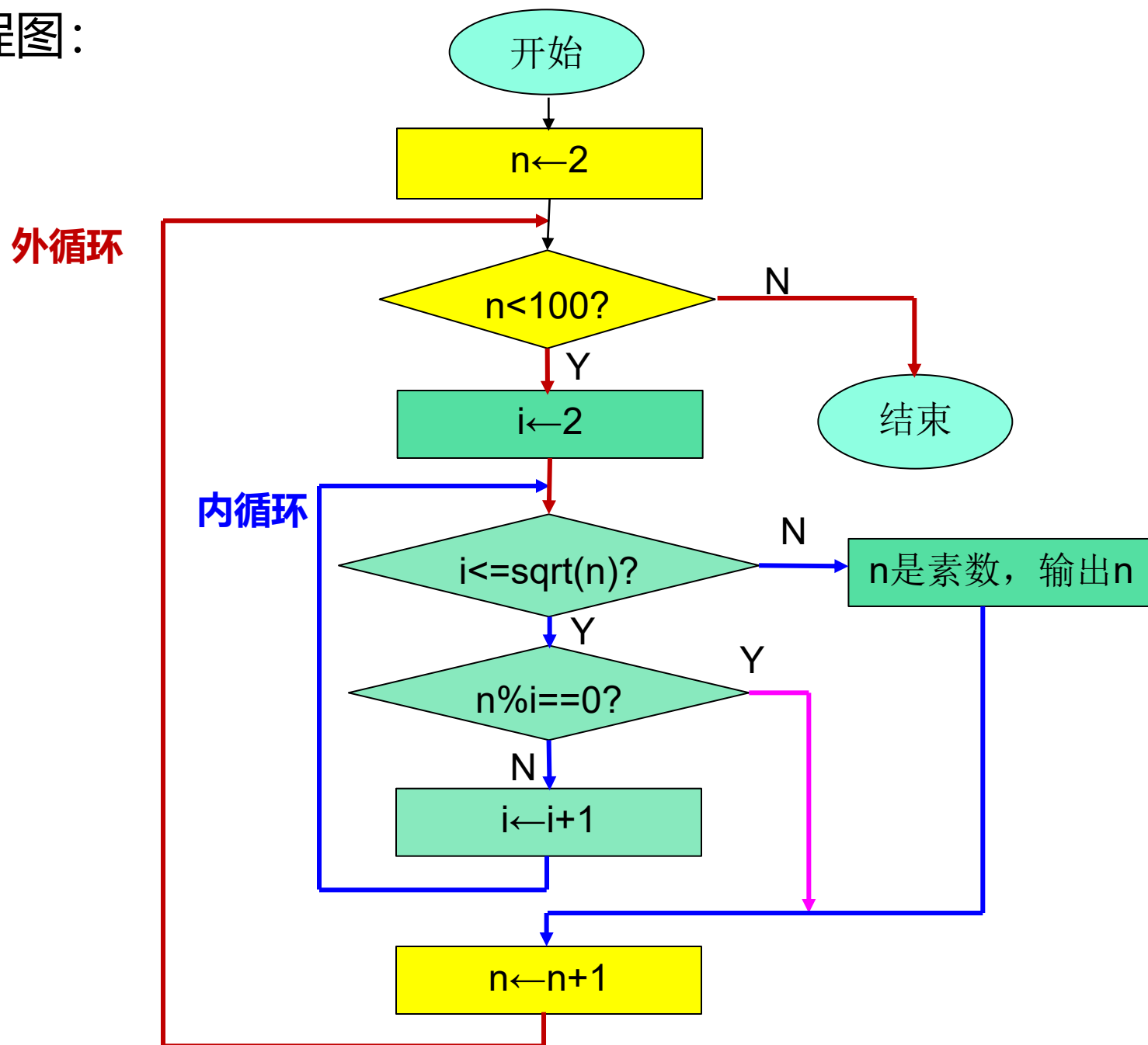
分析：

需要**双重嵌套的循环结构**：

外循环：设循环变量是n， $n=2\sim 100$ 。对每个n，判定是否为素数，是则输出，不是则立即测试下一个数。

内循环：测试当前的n是否为素数。设循环变量为i， $i=2\sim \sqrt{n}$ ，用i去除n。如果能整除，则n一定不是素数，就不要用后续的i除n了，马上可以**退出内循环**，回到外循环；而且，因为当前的n不是素数，所以不用输出n，而是**继续外循环的下一次迭代**，测试下一个数。

流程图：



与教材P105的程序略有不同

```
#include<stdio.h>
#include<math.h>
int main(void)
{
    int i, n, k, m;
    for (k=0, n=2; n<100; ++n){
        for (i=2; i<=sqrt(n); ++i)
            if( !(m = n % i) )
                break;
        if (!m) continue;
        printf("%6d", n);
        if ( !(++k % 10) ) printf("\n");
    }
    printf("\n");
    return 0;
}
```

//外循环

//内循环

//若i能整除n, n不是素数, 立即结束内循环

//若n不是素数, 外循环继续下一次迭代

//控制换行, 一行输出10个数

e:\test2\bin\Debug\test2.exe

2	3	5	7	11	13	17	19	23	29
31	37	41	43	47	53	59	61	67	71
73	79	83	89	97					

Process returned 0 (0x0) execution time : 0.031 s
Press any key to continue.

continue语句的使用要点:

- **continue语句只能出现在循环语句中。**

continue语句执行时, 跳过从continue语句开始到循环体结束之间的所有语句, 直接转移到循环的末尾:

- ◆ 对于**while (e) s;**和**do s while(e);**, 执行continue语句将立刻转向执行对循环控制表达式(**e**)的计算和测试;
- ◆ 对于**for (e1;e2;**e3**) s**, 则马上执行表达式**e3**, 然后执行对表达式e2的计算和测试。

break语句的使用要点:

- ◆ break语句有两处应用:

(1)用于switch语句中: 从case子句中退出switch语句;

(2)用于循环语句中: 从循环体内直接退出到循环体外。

- ◆ 对于嵌套的switch语句或循环语句, break语句的执行只能退出直接包含该break语句的那一层结构, 不能跨层。

例4.20 输入10个整数，输出其中正数的个数及平均值。

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int a, i, k, x;
```

```
    printf("input 10 numbers:\n");
```

```
    for (a=i=k=0; i<10; ++i) {
```

```
        scanf("%d", &x);
```

```
        if (x<=0)
```

```
            continue;           /* 对负数不作处理 */
```

```
            a+=x;                /* a是累加器，计算正数的和，存入a */
```

```
            ++k;                 /* k是计数器，统计正数的个数 */
```

```
    }
```

```
    if (k) printf("numbers=%d, average=%f\n", k, 1.0*a/k);
```

```
    return 0;
```

```
}
```

2. return语句

return语句的功能：结束**被调用函数**的执行，返回到主调环境中。如果当前是main函数，则结束程序的运行，返回操作系统。

return语句有下面两种形式：

(1) **不带返回值表达式的return语句：** **return;**

(2) **带返回值表达式的return语句：** **return 表达式 ;**

- ◆ 不带返回值表达式的return语句只作返回控制，没有返回值，用于void类型的函数；
- ◆ 带返回值表达式的return语句将先计算表达式的值，然后再返回，表达式的值将作为**函数的返回值**返回。

例4.24 写一个函数**sign**，对参数x，如果x小于0，则返回-1；如果x等于0，则返回0；如果x大于0，则返回1。

```
int sign(double x)
```

```
{
```

```
    if (x < 0)
```

```
        return -1;
```

```
    else
```

```
        return ( (!x) ? 0 : 1 );
```

```
}
```

外层括号可以省略：

```
return (!x) ? 0 : 1 ;
```

return语句的使用要点：

- ◆ 如果函数不是void类型的，则其中的所有return语句都要有表示返回值的表达式；且return返回值的类型应该和函数的类型匹配。
- ◆ 如果函数是void类型的，则其中所有return都不能带返回值。

3、goto语句和标号语句

- **goto语句**：又称为**无条件转移语句**，即立即、无条件地转向一个**标号语句**。

goto语句的一般形式为：**goto 标号**;

- **标号语句**：任何**可执行的C语句**前面都可以加“**标号**”前缀而成为**标号语句**。

标号语句的一般形式为：**标号：语句**

其中，标号是一个合法的标识符。

例 输入一个整数，如果是偶数，输出"偶数"， 否则输出"奇数"。

```
#include<stdio.h>
```

```
int main(void)
```

```
{
```

```
    int n;
```

```
    scanf("%d",&n);
```

```
    if(n%2) goto L1;
```

```
    else goto L2;
```

```
    L1: {
```

```
        printf("奇数");
```

```
        goto L3;
```

```
    }
```

```
    L2: printf("偶数");
```

```
    L3: 可以对“空行”定义标号语句
```

```
    return 0;
```

```
}
```

例4.22：输入一个算式，模拟袖珍计算器的加、减、乘、除四则运算。

- 假定计算时不考虑运算符的优先级，也不允许输入圆括号()，而是仅**按照运算符出现的先后顺序执行运算**。

例如， 输入：10.8+0.13*100

计算结果为：1093.000000

```
include <stdio.h>
int main(void)
{
    double x, y;
    char op;
inx:
    printf("input arithmetic expression:\n");
    scanf("%lf", &x);
    while((op = getchar()) != '\n') {
iny:
        scanf("%lf", &y);
        switch(op){
            case '+': x += y; break;
            case '-': x -= y; break;
```

```
            case '*': x *= y; break;
            case '/': if (y) x /= y;
                else { /* 除数为0，重新输入除数 */
                    printf("divisor is zero, input divisor again!\n");
                    goto iny;
                }
            break;
            default: /* 运算符非法，重新输入算式 */
                printf("illegal operator, input arithmetic expression again!\n");
                goto inx;
        }
    }
    printf("%lf\n", x);
    return 0;
}
```

标号和goto语句的使用要点：

- ◆ 标号只能定义在函数内（包括main函数），不能定义在全局空间（任何函数的外部）。
- ◆ 同一个函数内的标号不能重名，不同函数内的标号可以重名；
- ◆ 标号语句中的标号是**对标号的定义**，goto语句中的标号是**对标号的引用**。被引用的标号在程序里必须要有定义，但二者的先后顺序没有约束。一个goto语句只能引用一个标号，不能同时goto到多个标号语句；
- ◆ 标号只能定义一次，但可以被任意次引用。允许定义好的标号语句没有对应的goto语句。

◆ goto语句和其引用的标号必须在同一函数内，不能跨函数。

即goto语句不能从一个函数转移到另一个函数中。

```
void f1()
{
    .....
    L1: .....
    .....
}
void f2()
{
    .....
    goto L1;
    .....
    L1: .....
}
```

定义的标号没有用

对本函数内的标号的引用

```
void f1()
{
    .....
    L2: .....
    .....
}
void f2()
{
    .....
    goto L2;
    .....
    L1: .....
}
```

不能跨函数引用标号

使用goto语句的问题：

- ◆ **缺点：** 转向比较随意，**破坏程序的结构化特性**，容易使程序的逻辑结构不清，不便于维护。
- ◆ **优点：** 可以从深层**嵌套的结构**中直接转到外层，效率高。

结构化程序设计要求尽量少用或不用goto语句。

4.7 嵌套循环的程序设计

一个循环语句的循环体中又包含另一个循环语句的形式称为**嵌套循环**。又称为**多重循环**。

- **三种循环语句可以相互、任意嵌套。**
 - ◆ 两层嵌套的循环称为**二重循环**，n重嵌套的循环称为**n重循环**。
 - ◆ 二重及以上的嵌套循环统称为**多重循环**。
- C语言对循环的嵌套层数没有明确限制（C99最多允许循环嵌套127层），其中二重循环应用最为普遍，其次是三重循环。

例4.24 计算 $\text{sum}=1^1+2^2+3^3+\dots+n^n$ ， n 由终端输入。

分析：

- ◆ 先输入 n 。
- ◆ 设整型变量 i ， i 从1开始递增，直至 n 。控制计算 sum 的项数
- ◆ 对于每个 i ，计算 $\text{term}=i^i$ ：即 i 个 i 相乘
- ◆ **计算 sum ：二重循环：**
 - 内层循环，通常称为 “**内循环**”，计算 i 的累乘 term ；
 - 外层循环，通常称为 “**外循环**”，将 term 累加到求和变量 sum 上。

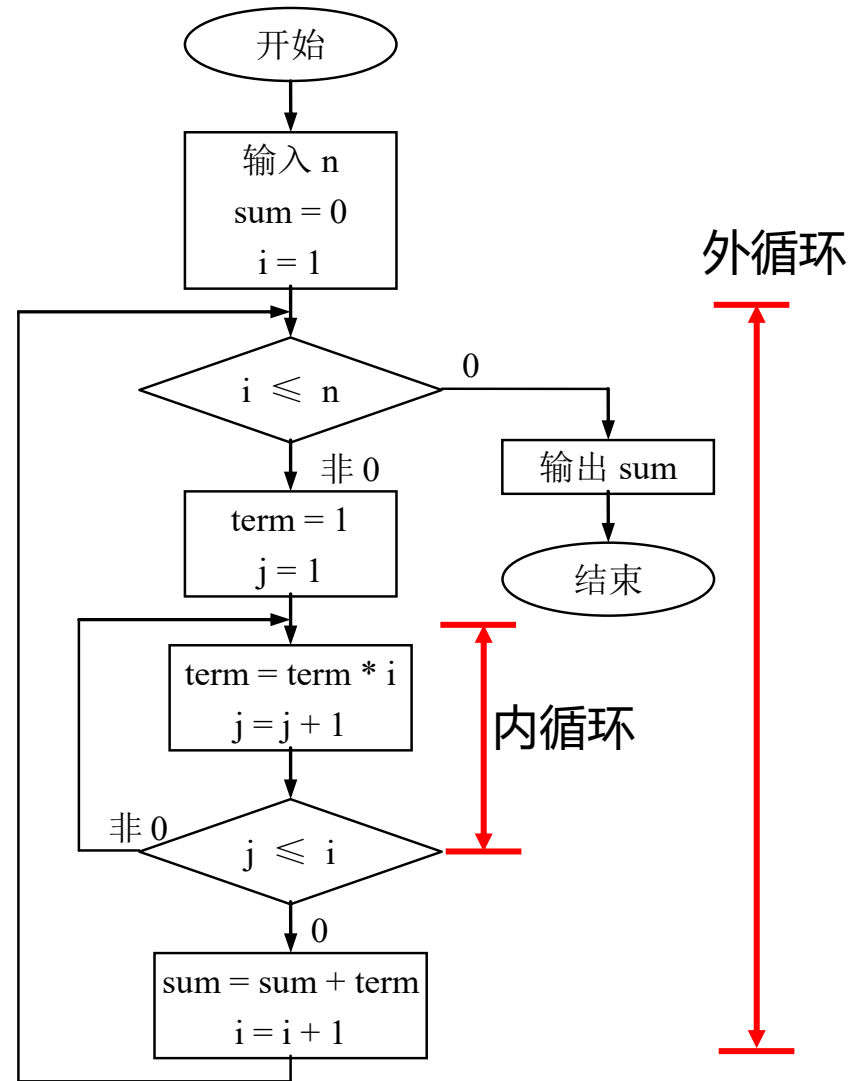
```

#include <stdio.h>
int main(void)
{
    int i, j, n;
    long s, term;
    printf("input n:\n");
    scanf("%d", &n);
    for (s=0, i=1; i <= n; ++i)
    {
        term = 1;
        j = 1;
        do
        {
            term *= i;
            ++j <= i;
            s += term;
        }
        printf("s=%ld\n", s);
        return 0;
    }
}

```

外循环

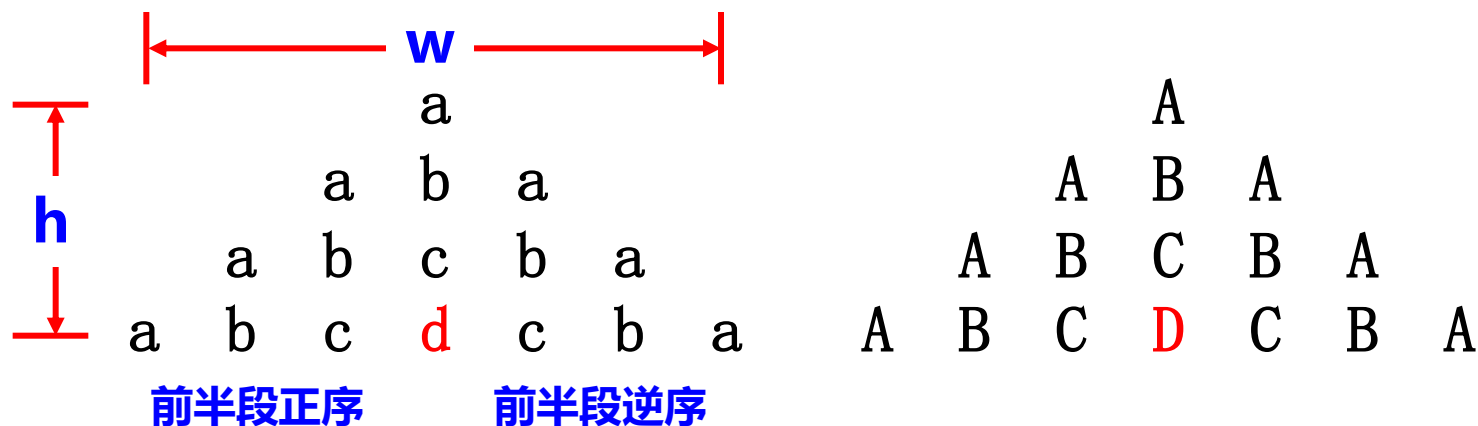
内循环



流程图

例4.25 输入一个字母，在**屏幕正中**输出由这个字母决定其高度的
"字符金字塔"。

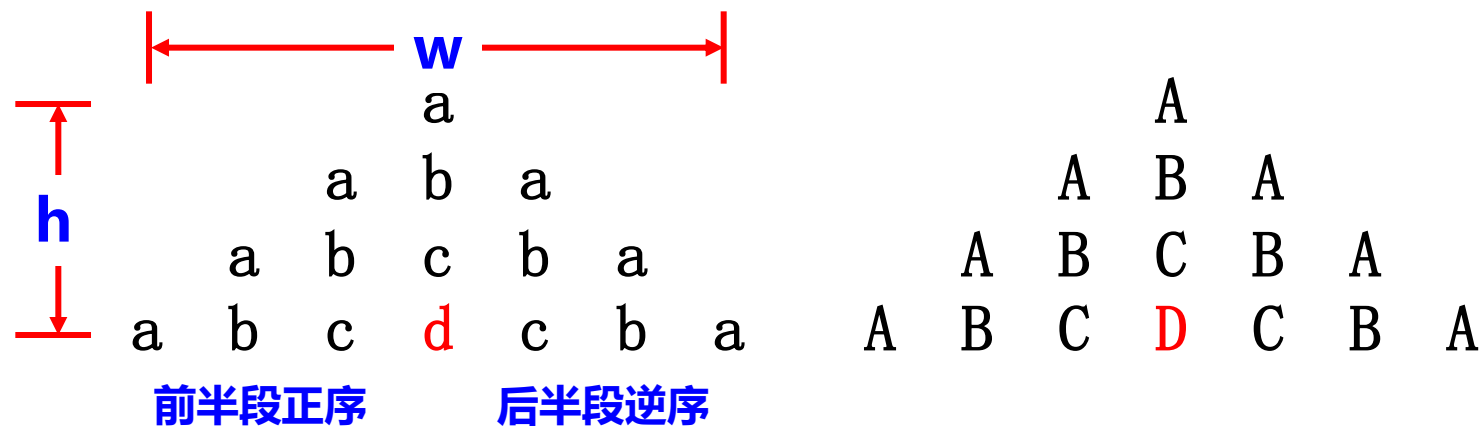
例如，输入小写字母 'd'，则输出下列左边的图形，如果输入
大写字母 'D'，则输出右边的图形。



分析: ① 设输入的字母存入字符变量 **c** 中;

② 金字塔的高度 **$h = c - 'a' + 1$** (或 **$c - 'A' + 1$** , 大小写),

宽度 **$w = 2 * h - 1$** ;



逐行输出金字塔的形状：前半段正序输出，后半段逆序输出

程序结构：双重循环

- ◆ 外循环控制高度，输出h行；
- ◆ 内循环输出一行宽度为w、左右对称的字符序列；
每行的左侧要先输出一定量的空格。

算法步骤:

(1) 输入字符 c ;

(2) 如果 c 是小写字母则置塔顶 top 为'a'，如果 c 是大写字母则置 top 为'A'，
如果 c 是非字母字符，则置 top 为'\0' (然后结束程序)；

(3) 如果 top 非0，则输出图形

(3-1) 置外循环变量 $c1$ 的初值为 top ('a'或'A');

(3-2) 如果 $c1 \leq c$ ，则输出一行：

(3-2-1) 输出一行左边的所有空格，空格数目为： $40-2*(c1-top)$ ；

(3-2-2) 输出一行的前半段 (包括正中的一个字符)；

(3-2-3) 输出一行的后半段；

(3-2-4) 输出换行： $c1=c1+1$ ，重复(3-2)，循环输出下一行，直至
 $c1 > c$ 则结束循环。

```
#include <stdio.h>
```

```
#include <ctype.h>
```

← isupper、islower: 判断是否为大写或小写字母

```
int main(void)
```

```
{
```

```
    char c, c1, c2, top;
```

```
    int i;
```

```
    printf("input a character:\n");
```

```
    top = isupper(c = getchar()) ? 'A' : (islower(c) ? 'a' : '\0'); //确定塔顶字母
```

```
    if (top) {
```

```
        for (c1=top; c1<=c; ++c1){
```

```
            for (i=1; i<=40-2*(c1-top); ++i) putchar(' '); //输出一行左边的空格
```

```
            for (c2=top; c2<=c1; ++c2) printf("%2c", c2); //输出一行的前半段字符
```

```
            for (c2=c1-1; c2>=top; --c2) printf("%2c", c2); //输出一行的后半段字符
```

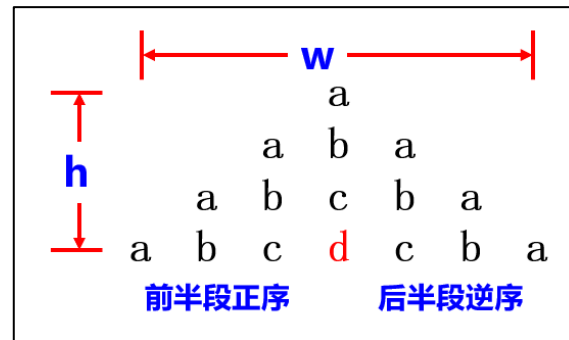
```
            printf("\n"); //换行
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```



注：多重循环中，**内、外循环的循环控制变量不能同名**，
否则会造成循环控制混乱，导致死循环或计算结果错误。

如：for (**i=0; i<n; i++**) {

for (**i=0; i<m; ++i**) { **×**

.....

}

}

4.8.2* 应用举例

1、枚举

一组参数当前的一组特定取值称为问题的一个**状态**，这个状态下的参数的值可能导致问题的解，也可能不导致问题的解。把**一组参数的所有可能取值的组合称为问题的状态空间**。

枚举：就是列举问题状态空间中的每一种可能状态，直到找到问题的解（如果存在话）。

枚举是一种简单、暴力的搜索算法。编程思想相对简单，通常用**循环语句+条件语句**来实现。

例4.27 下列乘法算式中，每个汉字代表1个0~9的数字，不同的汉字代表不同的数字。试编程确定使得整个算式成立的数字组合，如有多种情况，请给出所有可能的答案。

$$\text{赛软件} * \text{比赛} = \text{软件比拼}$$

分析：

- ◆ 这个算式中有**赛**、**软**、**件**、**比**、**拼**5个不同的汉字，设计程序时，分别用变量 **a**、**b**、**c**、**d**、**e** 来代表。
- ◆ 把算式中的“**赛软件**”理解为百位数、“**比赛**”理解为十位数、“**软件比拼**”理解为千位数，从而本题的算法就是对**a**、**b**、**c**、**d**、**e**的一组取值判定下式是否成立：

$$(a*100+b*10+c) * (d*10+a) == b*1000+c*100+d*10+e。$$

赛软件 * 比赛 = 软件比拼

$$(a*100+b*10+c) * (d*10+a) == b*1000+c*100+d*10+e。$$

由于每个变量的取值为0 ~ 9，每个变量的值都不同，所以这5个变量所有可能的取值是一个10取5的排列，一共有 $10*9*8*7*6=30240$ 种情况。

枚举算法：依次测试五个变量所有的取值组合，如果满足算式，则说明得到了问题一个解。直到测试完毕，得到所有解。

程序的结构：要对五个变量所有可能的取值进行组合和测试，所以需要有一个**五重循环结构**。

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int a, b, c, d, e;
```

```
    for (a=0; a<10; a++)
```

/* 第一层循环执行10次，测试a取值0~9的情况 */

```
        for (b=0; b<10; b++) {
```

/* 第二层循环执行10次，测试b取值0~9的情况 */

```
            if (b-a==0) continue;
```

/* 判定b是否等于a，等于a的值不用测试*/

```
            for (c=0; c<10; c++) {
```

```
                if ((c-b)*(c-a) == 0) continue; /* (c-b)*(c-a) == 0等价于(c-b)==0 || (c-a)==0, 下同 */
```

```
                for (d=0; d<10; d++) {
```

```
                    if ((d-c)*(d-b)*(d-a) == 0) continue;
```

```
                    for (e=0; e<10; e++) {
```

```
                        if ((e-d)*(e-c)*(e-b)*(e-a) == 0) continue;
```

```
                        if ((a*100+b*10+c) * (d*10+a) == b*1000+c*100+d*10+e)
```

```
                            printf("%d%d%d * %d%d = %d%d%d%d\n", a, b, c, d, a, b, c, d,e);
```

```
                    } } } }
```

```
    return 0;
```

```
}
```

五重循环

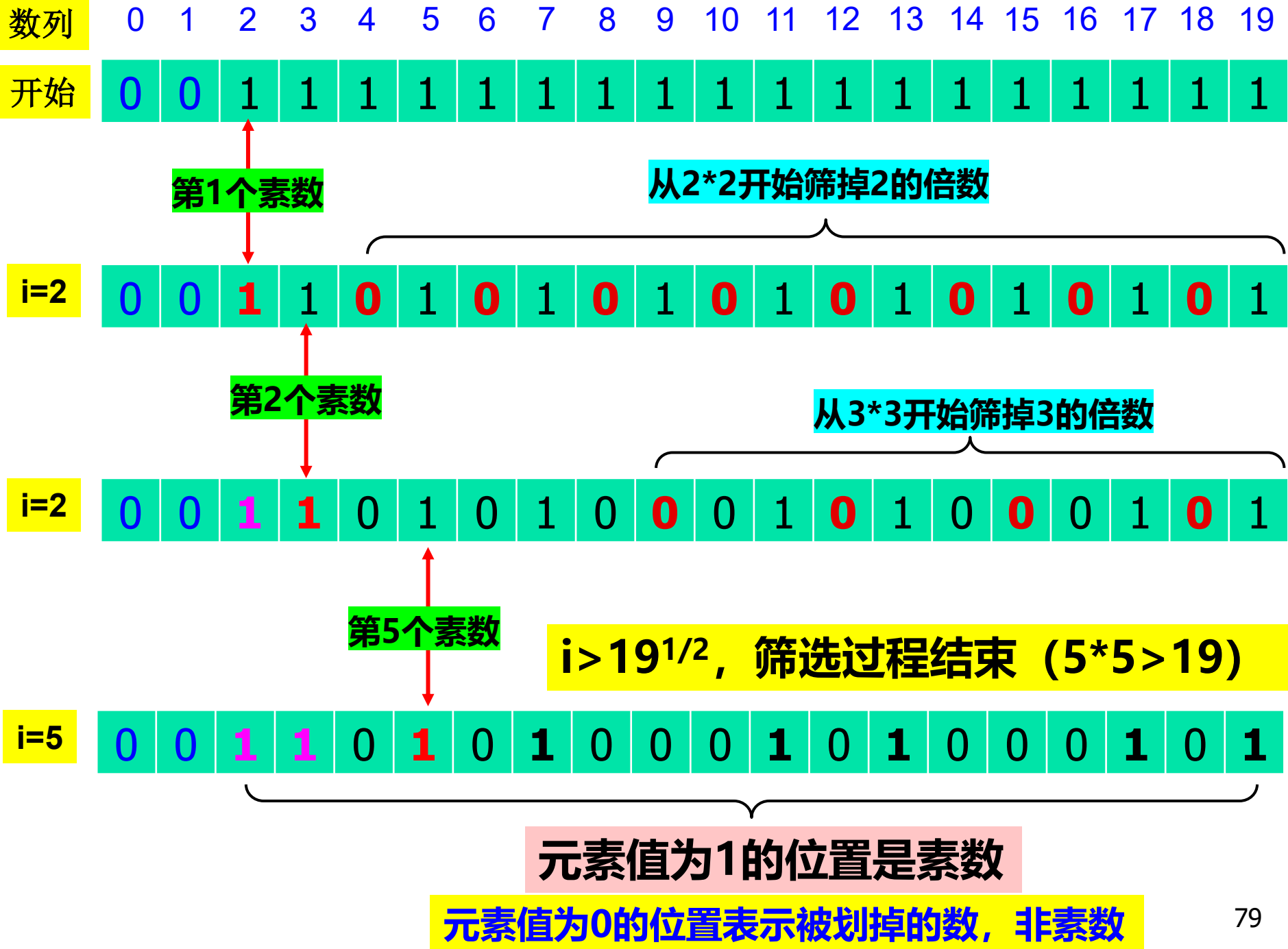
2、筛选

筛选法： 去掉那些不可能是问题解的值，剩下的就是问题的解。

例： **埃拉托斯特尼筛选法**是一种构造素数表的有效方法。

具体做法是，对1 ~ n之间的整数：

- ◆ **1**既非素数又非合数，将1划掉；
- ◆ **2**是第一个素数，而2的倍数都不是素数，所以将2后面所有2的倍数都划掉。再从2往后搜索，重复以下操作：
 - **找后面第一个未被划掉的数，该数是素数**，然后划掉该数后面是它的整数倍的数。再找下一个未被划掉的数重复此操作，直到n（实际 $\sqrt{n}$ ）为止。
- ◆ **序列中最后未被划掉的数就是2~n之间的所有素数。**



例4.28 用筛选法构造2 ~ 100之间的素数表。

```
#include <stdio.h>
```

```
int main()
```

```
{ int i, j, a[100];
```

数组a是**标志数组**，a[i]=0 表示下标i不是素数，1表示i是素数。初始时所有的a[i]=1，筛选过程就是将不是素数的i对应的元素a[i]置为0。

```
for (a[0]=a[1]=0, i=2; i<100; i++) a[i] = 1;
```

```
for (i=2; i<50; i++)
```

```
if (a[i]) for (j=i*i; j<100; j+=i) a[j] = 0;
```

核心操作：从2开始，筛掉每个素数的倍数。

```
for (i=2, j=0; i<100; i++) /* 输出素数表 */
```

```
if (a[i]) {
```

```
printf("%4d", i);
```

```
if (++j == 8)
```

```
{ j = 0; printf("\n"); }
```

```
}
```

/* 每行输出8个素数 */

```
return 0;
```

```
}
```

e:\test2\bin\Debug\test2.exe

```
2   3   5   7  11  13  17  19
23  29  31  37  41  43  47  53
59  61  67  71  73  79  83  89
97
```

Process returned 0 (0x0) execution time: 0.000 s
Press any key to continue.

3、递推

递推：从初始状态开始，按照一定的规则设法一步步推导，直到得到问题的解。

例4.29 取球游戏。盒子里有 n ($n < 10000$) 个球，A、B两人轮流从盒中取球，约定的游戏规则为：

- (1) 每次从盒子中取出的球的数目必须是**1, 3, 7**或者**8**个；
- (2) **A先取球, B接着取球**，如此双方交替取球，直到盒里的球被取完；
- (3) **取走盒中最后一个球的一方为输方。**

假设取球过程中A和B都不会失误。编程实现，输入小球的数目 n ，按规则取球，如果A为赢方则输出1，A为输方则输出0。

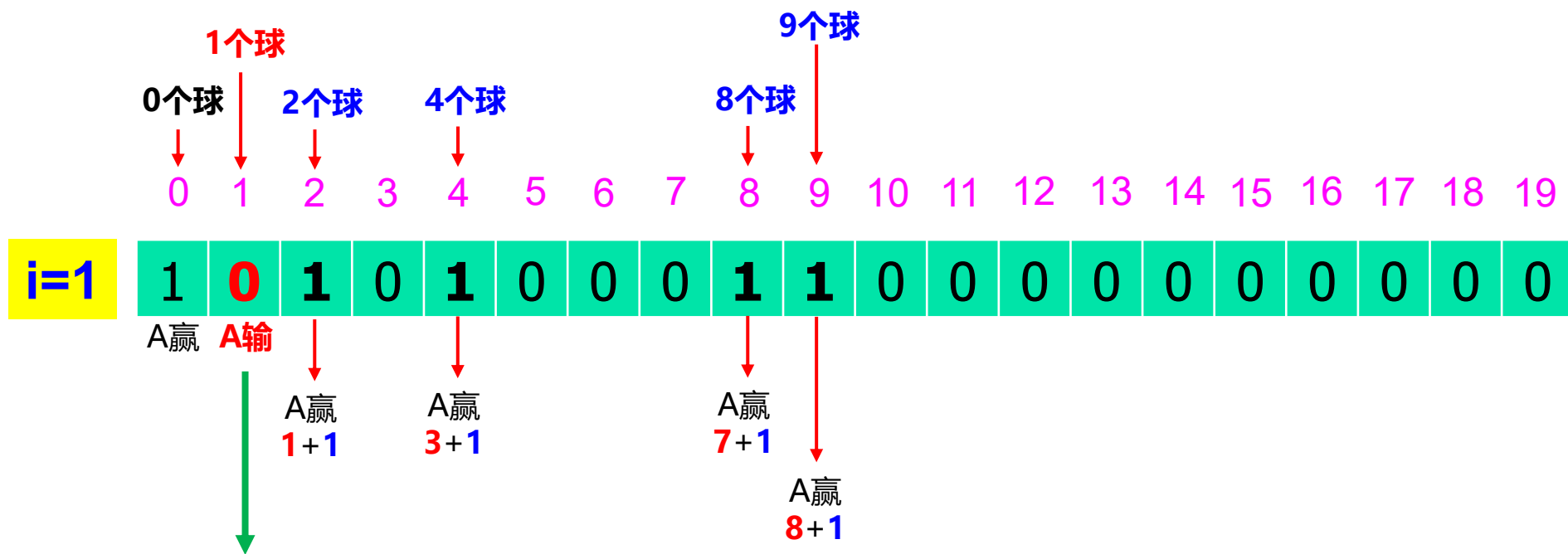
分析（模拟取球的过程）：

n=0：无球可取，相当于**A赢**；

n=1：只有一个球而A必取，所以A必输。相当于只有1个球时，谁先取随输。

但据此可得：当有**1+1、1+3、1+7、1+8**个球时，B必输（A先手，拿1、3、7、8个球，剩下一个球给B，B必输）。

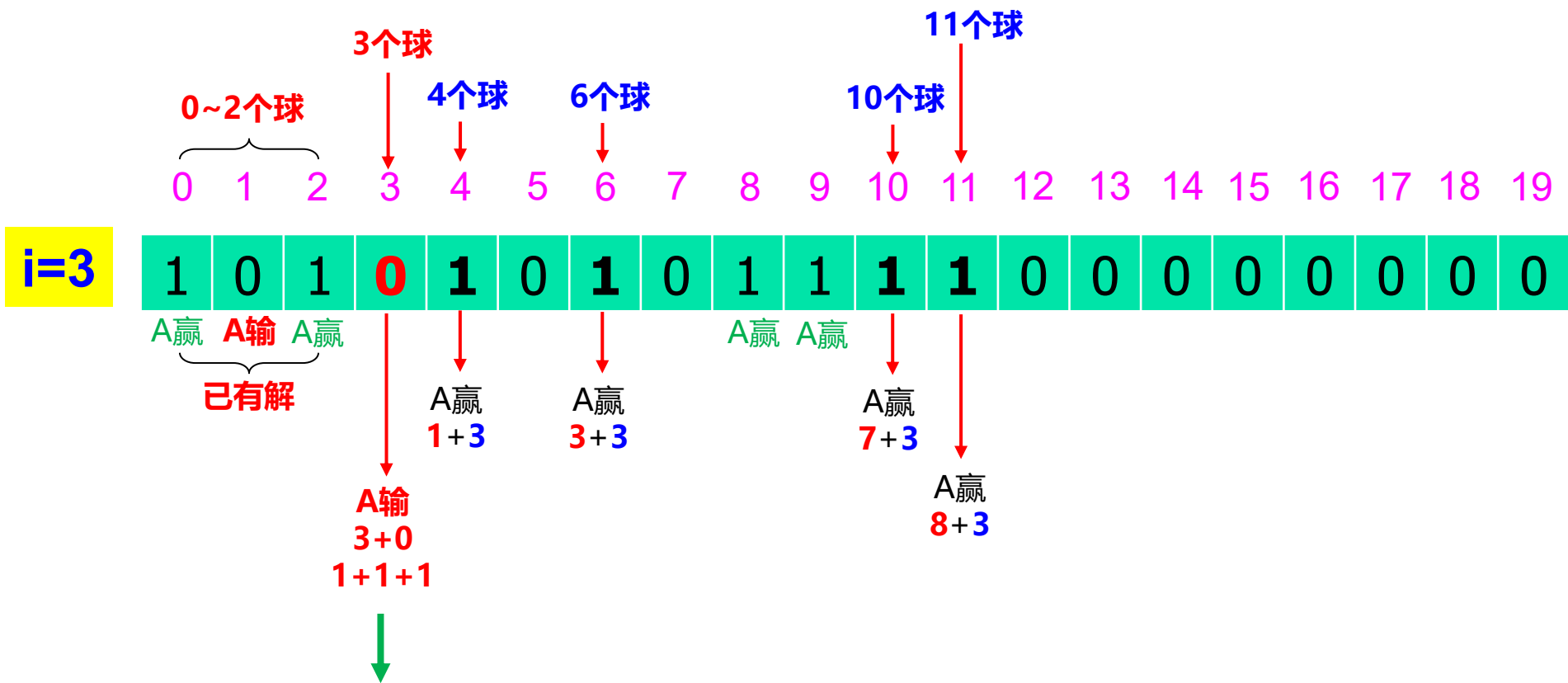
如下，用数组A表示不同球数时A的输赢（0：A输，1：A赢）。



从**n=1、A输**，并可以推导出**n=2、4、8、9**时，**A可以赢**。

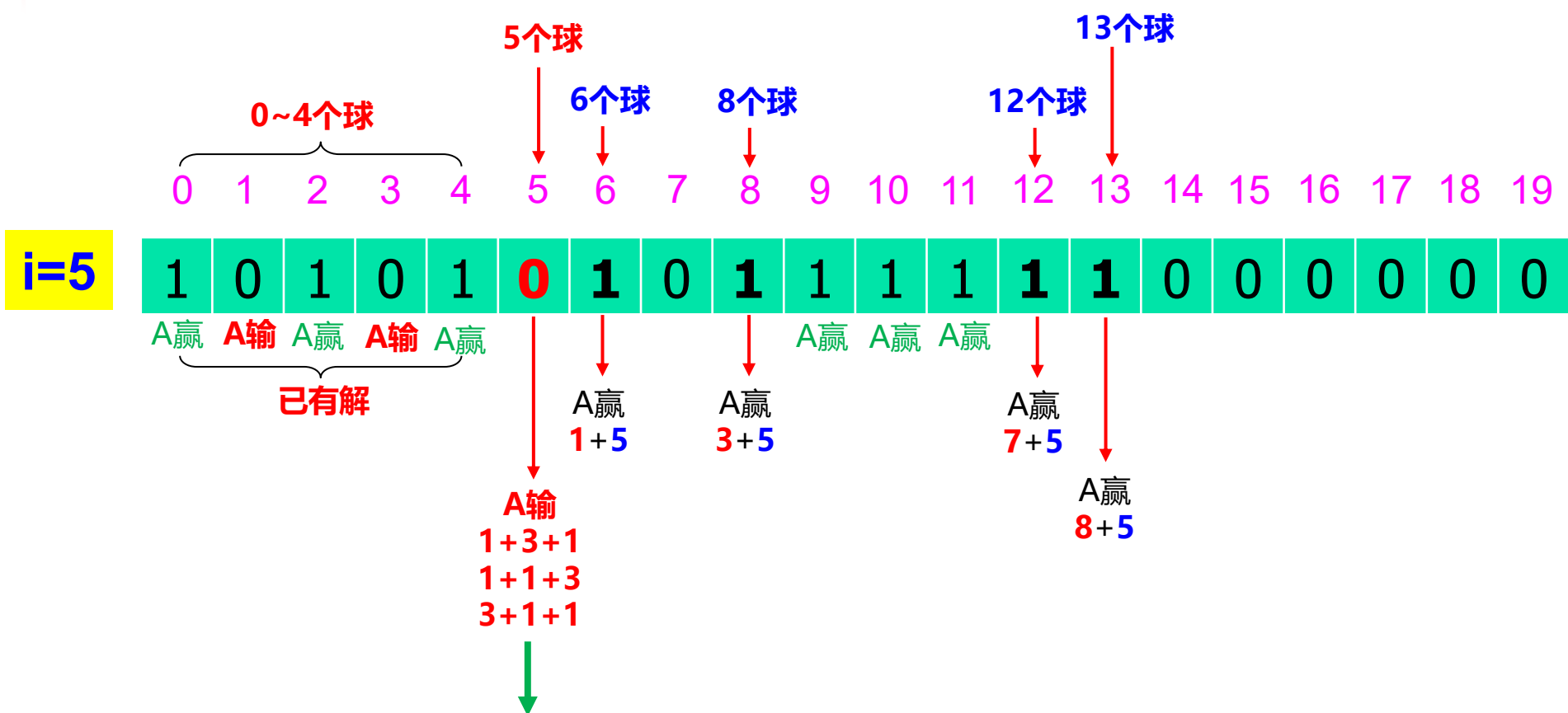
继续分析：能保证A赢的位置不用多考虑，重点看A会输的位置。

下一步看下一个会导致A输的位置是A[3]=0，但据此，当有3+1、3+3、3+7、3+8个球时，A必赢（A先手，只要不失误，拿1、3、7或8个球，剩下3个球必导致B输），如下：



从n=3、A输，可以推导出n=4、6、10、11时，A可以赢。

再继续：再找下一个会导致A输的位置，**A[5]=0**，同样，当有**5+1、5+3、5+7、5+8**个球时，A必赢（A先手，只要不失误，拿1、3、7或8个球，剩下5个球必导致B输）：



从**n=5、A输**，可以推导出**n=6、8、12、13**时，**A可以赢**。

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

i=1 1 0 1 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0

i=3 1 0 1 0 1 0 1 0 1 1 1 1 0 0 0 0 0 0 0 0

i=5 1 0 1 0 1 0 1 0 1 1 1 1 1 1 0 0 0 0 0 0

i=7 1 0 1 0 1 0 1 0 1 1 1 1 1 1 1 1 0 0 0 0

持续搜索：**i=1、3、5、...**，即：

从一个 **$A[i]=0$ 的位置**起，可以确定其后的四个元素：

$A[i+1]$ 、 $A[i+3]$ 、 $A[i+7]$ 、 $A[i+8]$ 等于1。

对一个足够大数组 **$A[0\sim N]$** ，**事先按此计算好每个元素的值**。

然后，**输入一个n**， $n \leq N$ ，对A “**查表**”， $A[n]$ 的值就是问题的

解： **$A[n]=0$ 表示A输， $A[n]=1$ 表示A赢**。

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    char tab[10000];
```

```
    int ins[ ] = {1, 3, 7, 8};
```

增量值

```
    int i, j, n, num;
```

```
    for (tab[0]=1, i=1; i<10000; i++) tab[i] = 0;
```

```
    for (i=1; i<10000; i++) {
```

```
        if (tab[i] == 0)
```

```
            for (j=0; j<4; j++)
```

```
                if ((num = i + ins[j]) < 10000)
```

```
                    tab[num] = 1;
```

```
    }
```

```
    scanf("%d", &n); /* 输入当前的球数*/
```

```
    printf("%d\n", tab[n]);
```

查表：直接查tab（tab中有所有球数情况下的解），

```
    return 0;
```

其中**tab[n]**就是有n个球时的解。

```
}
```

对有0~9999个球的所有情况进行计算。

——**造表**

***例4.10** 输入一个C程序(一段正文)，按原来格式复制输出，复制

自学 过程中删去输入程序中所有的注释。

注释： 程序正文中处于`/*...*/`之间的文本称为**注释**。

多行注释

如： `int main(void)`

`{`

`char c;     /* 定义字符变量c */`

`printf("input a text end of ctrl + z:\n");`

`c = getchar();     // 从标准输入设备获取字符`

`return 0;`

单行注释 `//`: C++

`}`

注释是程序员对程序的解释和说明，是给人看的，而不是程序的必备部分。编译器在编译程序时，也是先**将注释过滤掉**，然后再对没有注释的**纯程序**进行编译。

```
#include <stdio.h>

int main(void)
{
    char c; /* 定义字符变量c */
    printf("input a text end of ctrl + z:\n");
    c = getchar(); /* 从标准输入设备获取字符*/
    return 0;
}
```

有注释的程序

过滤注释



```
#include <stdio.h>

int main(void)
{
    char c;
    printf("input a text end of ctrl + z:\n");
    c = getchar();
    return 0;
}
```

过滤以后，没有注释的程序

好习惯：一般，应尽可能多加一些注释，以便于阅读和维护程序

怎么过滤程序？

首先要识别并区分出程序中的注释部分和非注释部分。非注释部分保留，注释部分删除。

建立一种**基于状态转换**的过程来处理：

◆ 将复制过程划分为4种状态：

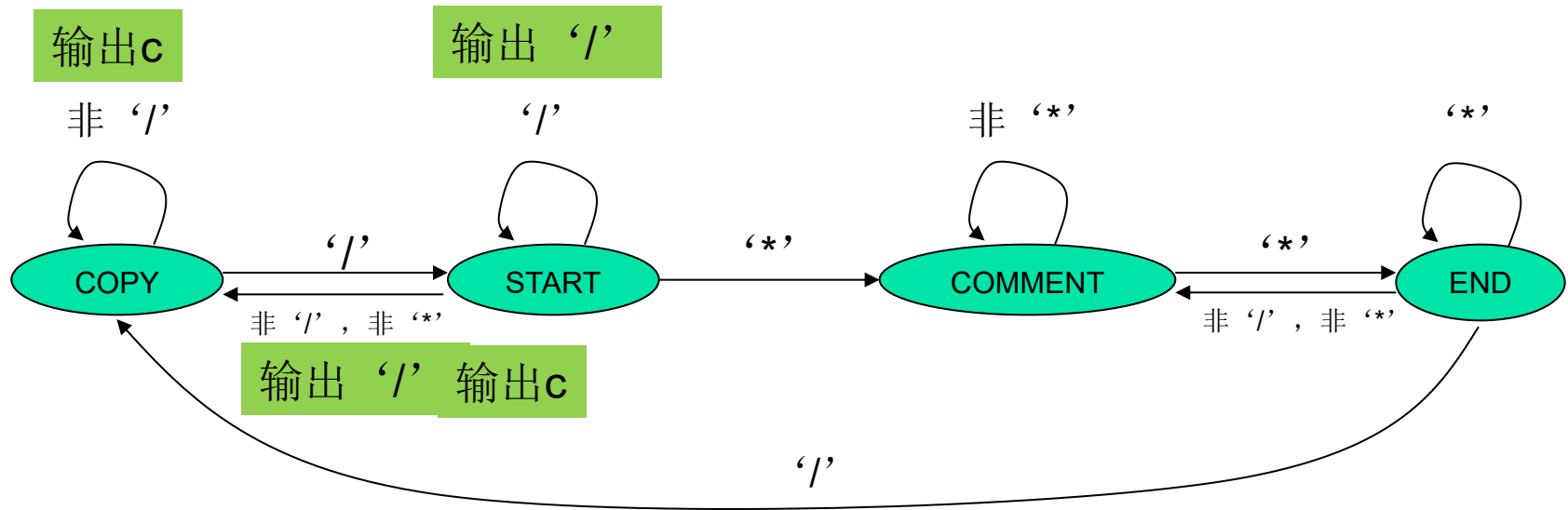
- 复制状态：**COPY**
- 开始注释状态：**START**
- 注释状态：**COMMENT**
- 结束复制状态：**END**

- 用数值0、1、2、3分别代表四个状态。
- 设置状态变量：**int state;**
- 开始的时候，state的初值为COPY。

◆ 程序的状态会因为输入不同的字符而发生转换，如下：

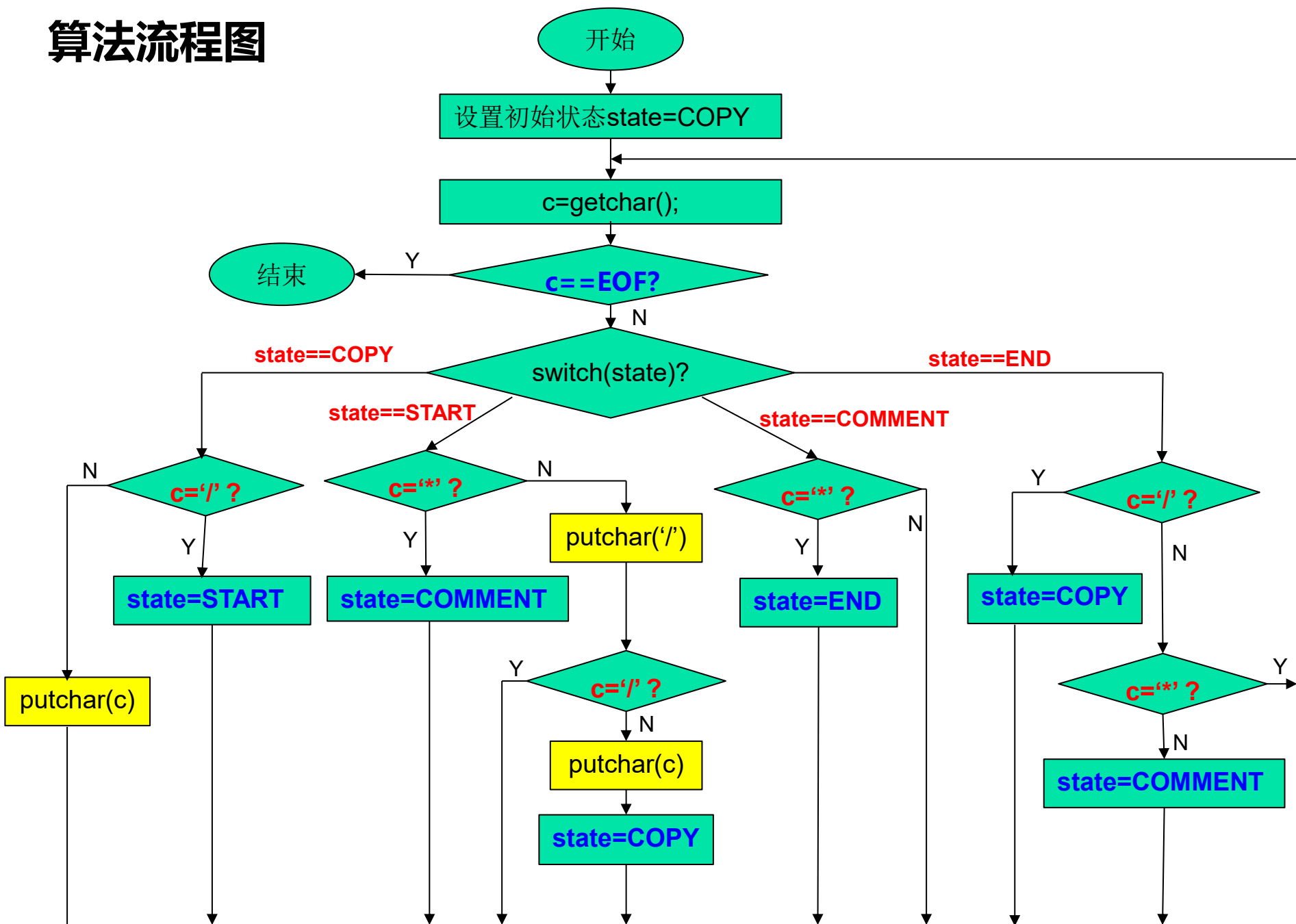
z=x/y*2; /* 计算z=x/y*2 **/

从左向右扫描文本中的字符序列



状态转换图

算法流程图



```

#include <stdio.h>

enum {COPY, START, COMMENT, END};

int main(void)
{
    char c;
    int state = COPY;
    printf("input C program end with ctrl+z:\n");
    while ((c=getchar()) != EOF)
        switch (state) {
            case COPY:  if (c == '/')  state = START;
                        else  putchar(c);
                        break;
            case START: if (c == '*')  state = COMMENT;
                        else {
                            putchar('/');
                            state = (c == '/') ? START : (putchar(c), COPY);
                        }
                        break;
            case COMMENT: if (c == '*')  state = END;
                        break;
            case END:  state = (c == '/') ? COPY : ((c == '*') ? END : COMMENT);
        }
        /* end of switch, end of while */
    return 0;
}

```

初始状态为COPY。每种状态下的处理方法如下：

(1) 在COPY状态下，若读入的字符为 '/' (可能为注释开始符号)，则将状态改为START；否则将读入的字符复制到输出。

(2) 在START状态下，若读入的字符为 '*' (确定注释开始)，则将状态改为COMMENT；否则(不是注释)，将上一次读入的字符 '/' 复制到输出，并保持当前状态为START不变；

若当前读入的字符为 '/'，则保持当前状态为START不变；否则为正常字符，输出字符并将状态恢复成COPY。

(3) 在COMMENT状态下，若读入的字符为 '*' (可能为注释结束符号)，则将状态改为END。

(4) 在END状态下，若读入的字符为 '/' (确定注释结束)，则将状态恢复成COPY；否则(不是注释结束)；如果读入的字符是 '*'，则状态保持END不变，否则将状态改为COMMENT。

本章小结

- C语言中**表达式**和**语句**的概念。
- 结构化程序的三种基本结构：**顺序、分支、循环**。
- **分支语句**：if语句，switch语句
- **循环语句**：while语句、for语句和do-while语句。
- **转移语句**：break语句、continue语句、return语句、goto语句（标号语句）
- 掌握一些**基本的算法**，比如，**计算日期、判断闰年、解方程、处理字符、求阶乘、判断素数、输出二维字符图形**等。以及**枚举、筛法、递推**等算法设计的基本策略。